Chiri Engineering Assessment — AI Document Editor

The Challenge

Build a **collaborative AI document editor** in the browser. The user writes Markdown, and an AI agent works alongside them — suggesting edits, showing diffs, and iterating on the document together. Don’t build a full markdown editor from scratch, a prebuilt like tiptap or similar is fine.

Think Google Docs meets AI pair-writing. The user is always in control, but the AI is a capable collaborator that can rewrite paragraphs, fix tone, restructure sections, or follow specific instructions.

What to Build

A single page web app, no auth required, where:

1. The user can **write and edit Markdown** in a browser-based editor
2. An AI agent can **suggest changes** to the document (or a selection)
3. Suggestions appear as **visible diffs** — the user can see exactly what the AI wants to change before accepting
4. The user can **accept, reject, or refine** the AI's suggestions
5. The experience feels **collaborative** — like working with a smart co-author, not a chatbot in a sidebar and not something that completes the whole document

Beyond that, the design is yours. We're intentionally leaving this open and how you interpret "collaborative editing with AI" tells us a lot.

What We Provide

- An **OpenRouter API key** with a $5 spending cap — use any model available through OpenRouter
- That's it. The rest is up to you.

What to Submit

1. **Host your code** on GitHub or GitLab and send us the link
2. **Include a README** with: how to run it, what you built, and what you'd do differently with more time

3. **Save your AI session(s)** — we want to see how you used AI to build this (Claude transcript, Cursor composer history, ChatGPT thread, etc.). This is not optional, please commit this to the git repository along with the project code.

What We Evaluate

What matters	**What we look for**
The experience	Is it intuitive? Can we open it, write something, and get useful AI suggestions without reading docs?
AI integration quality	Does the agent feel like a collaborator or a gimmick? Are diffs clear and useful?
How you used AI to build it	Did you use AI tools effectively to ship faster? Can you explain the choices made?
Code quality	Is the code organized, readable, and something a teammate could pick up?
Taste & judgment	What did you prioritize? What did you intentionally leave out? Trade-offs matter more than features.

What We Don't Care About

- Docker, databases, auth systems — don't add infrastructure you don't need, this is just a showcase of a simple app so we can judge AI usage
- Visual perfection — but it should be usable and not broken
- Using a specific framework — React, Vue, Svelte, vanilla JS, whatever you're fast with

Guidelines

Time: Spend what feels right. This is meant to be fun, not a grind. A focused 4-6 hour session beats a sprawling weekend project.

AI usage is required, not just encouraged. We use AI every day in our engineering workflow. Show us how you work with it:

- Use Claude, Cursor, Copilot, ChatGPT, or any AI tool to build this
- Save the session/transcript — we'll review it alongside the code

Ideas (not requirements)

These are just sparks if you want them. Build whatever version of this excites you:

- Inline suggestions that appear as tracked changes
- A command palette or slash commands for AI actions
- Split-pane diff view (before/after)
- "Rewrite this section in a different tone" with live preview
- Version history showing how the document evolved with AI help
- Multi-turn refinement — "make it shorter," "now more formal," "add examples"